

# ■ Docker Command Cheat Sheet

CLI Commands • Docker Compose YAML • With and Without YAML explained

## ■ BUILD — Create your image from a Dockerfile

|                                                               |                                                                             |
|---------------------------------------------------------------|-----------------------------------------------------------------------------|
| <code>docker build -t myapp:1.0 .</code>                      | Build image from Dockerfile in current folder. Tag as myapp version 1.0     |
| <code>docker build -f Dockerfile.custom -t myapp:1.0 .</code> | Build using a specific Dockerfile name (not the default)                    |
| <code>docker build --no-cache -t myapp:1.0 .</code>           | Force fresh build — ignores cached layers. Use when changes aren't applying |
| <code>docker build --progress=plain -t myapp:1.0 .</code>     | Show full build output line by line — great for debugging failures          |
| <code>docker build --build-arg KEY=val -t myapp:1.0 .</code>  | Pass build-time variable into Dockerfile ARG (e.g. proxy settings)          |

## ■ RUN — Start a container from an image

|                                                            |                                                                   |
|------------------------------------------------------------|-------------------------------------------------------------------|
| <code>docker run -d --name myapp myapp:1.0</code>          | Run container in background (-d = detached) with a friendly name  |
| <code>docker run -d -p 8080:80 myapp:1.0</code>            | Map host port 8080 → container port 80. Access via localhost:8080 |
| <code>docker run --env-file .env myapp:1.0</code>          | Load all credentials and config from a .env file                  |
| <code>docker run -e KEY=value myapp:1.0</code>             | Pass a single environment variable inline                         |
| <code>docker run -v /host/path:/app/data myapp:1.0</code>  | Mount a host folder into the container (persistent data)          |
| <code>docker run --restart unless-stopped myapp:1.0</code> | Auto-restart if container crashes or server reboots               |
| <code>docker run --dns 8.8.8.8 myapp:1.0</code>            | Set custom DNS server for the container                           |
| <code>docker run --network host myapp:1.0</code>           | Use host network directly — no port mapping needed                |
| <code>docker run --rm myapp:1.0</code>                     | Run and auto-delete container when it exits (great for testing)   |

## ■■ MANAGE — Control containers and images

|                                                |                                                      |
|------------------------------------------------|------------------------------------------------------|
| <code>docker ps</code>                         | List currently running containers                    |
| <code>docker ps -a</code>                      | List ALL containers including stopped ones           |
| <code>docker start myapp</code>                | Start a stopped container                            |
| <code>docker stop myapp</code>                 | Gracefully stop a running container                  |
| <code>docker restart myapp</code>              | Stop then start a container                          |
| <code>docker restart \$(docker ps -q)</code>   | Restart ALL running containers at once               |
| <code>docker start \$(docker ps -aq)</code>    | Start ALL containers (including stopped)             |
| <code>docker rm myapp</code>                   | Delete a stopped container                           |
| <code>docker images</code>                     | List all images stored locally                       |
| <code>docker images   grep myapp</code>        | Filter images by name                                |
| <code>docker rmi myapp:1.0</code>              | Delete an image from local storage                   |
| <code>docker pull nginx:latest</code>          | Download an image from Docker Hub without running it |
| <code>docker tag myapp:1.0 myapp:latest</code> | Add another tag to an existing image                 |

## ■ DEBUG — Inspect and troubleshoot

|                                                                       |                                                               |
|-----------------------------------------------------------------------|---------------------------------------------------------------|
| <code>docker logs myapp</code>                                        | Show output/logs from a container                             |
| <code>docker logs -f myapp</code>                                     | Follow live logs in real time (like tail -f)                  |
| <code>docker logs --tail 50 myapp</code>                              | Show only last 50 lines of logs                               |
| <code>docker exec -it myapp bash</code>                               | Open an interactive shell inside a running container          |
| <code>docker exec myapp cat /etc/os-release</code>                    | Run a single command inside a container without a shell       |
| <code>docker inspect myapp</code>                                     | Show all details: IP, mounts, env vars, health status         |
| <code>docker inspect myapp --format='{{.State.Health.Status}}'</code> | Extract just the health check status                          |
| <code>docker stats</code>                                             | Live CPU, memory and network usage for all running containers |
| <code>docker top myapp</code>                                         | Show running processes inside a container                     |
| <code>docker diff myapp</code>                                        | Show files changed inside the container vs the image          |

## ■ CLEAN UP — Free disk space

|                                     |                                                                     |
|-------------------------------------|---------------------------------------------------------------------|
| <code>docker system prune</code>    | Remove stopped containers, unused networks and dangling images      |
| <code>docker system prune -a</code> | Full clean — also removes unused images. Frees lots of disk space   |
| <code>docker volume prune</code>    | Remove unused volumes. Careful — data inside is permanently deleted |
| <code>docker image prune</code>     | Remove only dangling (untagged) images                              |
| <code>docker container prune</code> | Remove all stopped containers                                       |
| <code>docker system df</code>       | Show how much disk space Docker is using                            |

## ■ DOCKER COMPOSE — Multi-container management with YAML

Instead of typing a long docker run command with many flags every time, you write all settings once in a docker-compose.yml file and run one short command. Think of it as saving your recipe so anyone can cook the same dish.

```
# docker-compose.yml – write once, run with: docker compose up -d
```

### **services:**

```
myapp: # service name
```

### **build:**

```
context: . # folder with Dockerfile
```

```
dockerfile: Dockerfile # which Dockerfile to use
```

```
image: myapp:1.0 # OR pull a ready image from Hub
```

```
container_name: myapp # friendly container name
```

```
restart: unless-stopped # auto-restart on crash/reboot
```

### **ports:**

```
- "8080:80" # host:container port mapping
```

### **env\_file:**

```
- .env # load credentials from .env file
```

```
environment: # OR set variables inline
```

```
- APP_ENV=production
```

```
- DEBUG=false
```

### **volumes:**

```
- mydata:/app/data # named volume (persistent)
```

```
- ./logs:/app/logs # bind mount (folder on machine)
```

### **dns:**

```
- 8.8.8.8 # custom DNS server
```

```
healthcheck: # verify app is actually working
```

```
test: ["CMD", "curl", "-f", "http://localhost"]
```

```
interval: 30s
```

```
retries: 3
```

### **logging:**

```
driver: "json-file"
```

```
options:
```

```

max-size: "50m" # rotate logs over 50 MB
depends_on:
- db # start db container first
db: # second service e.g. database
image: postgres:15
env_file: .env
volumes:
- pgdata:/var/lib/postgresql/data
volumes: # declare named volumes here
mydata:
pgdata:

```

## Docker Compose Commands

|                                             |                                                                |
|---------------------------------------------|----------------------------------------------------------------|
| <code>docker compose up -d</code>           | Build (if needed) and start all services in background         |
| <code>docker compose up -d --build</code>   | Force rebuild image then start — use after changing Dockerfile |
| <code>docker compose down</code>            | Stop and remove all containers defined in the file             |
| <code>docker compose down -v</code>         | Also delete volumes. Careful — this permanently removes data   |
| <code>docker compose restart</code>         | Restart all services at once                                   |
| <code>docker compose restart myapp</code>   | Restart only one specific service                              |
| <code>docker compose ps</code>              | Show status of all services in this compose file               |
| <code>docker compose logs -f</code>         | Follow live logs from all services                             |
| <code>docker compose logs -f myapp</code>   | Follow logs for one specific service only                      |
| <code>docker compose exec myapp bash</code> | Open shell inside a running service container                  |
| <code>docker compose pull</code>            | Pull latest versions of all images in the compose file         |
| <code>docker compose config</code>          | Validate and preview the final merged compose config           |

## ■ CLI vs COMPOSE — When to use which

| Use CLI (no YAML) when...                                                                                                                                                                                                                                                                                                                            | Use Docker Compose (YAML) when...                                                                                                                                                                                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• Running a quick one-off container to test something</li> <li>• Pulling and running a single image from Docker Hub</li> <li>• Debugging — checking logs, exec into containers</li> <li>• You only have one simple container with few options</li> <li>• Learning Docker basics for the first time</li> </ul> | <ul style="list-style-type: none"> <li>• You run the same container regularly with many flags</li> <li>• You have 2+ containers that work together (app + db)</li> <li>• You want to share config with your team via Git</li> <li>• Production or staging environment setup</li> <li>• You need volumes, DNS, health checks, networks</li> </ul> |

## Same thing — two ways:

|                 |                      |
|-----------------|----------------------|
| CLI way (long): | Compose way (short): |
|-----------------|----------------------|

```
docker run -d --name myapp -p 8080:80 --env-file  
.env --restart unless-stopped --dns 8.8.8.8  
myapp:1.0
```

*All flags typed every single time*

```
docker compose up -d
```

*All settings live in docker-compose.yml — run once*

■ ZPA Connector tip — since your setup needs restart, DNS, proxy, health check and env vars all together, Docker Compose is the right choice. One docker compose up -d command does everything.